

HO CHI MINH CITY UNIVERSITY OF
TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Course: Web Programming
ID: CO3050
Class: CC02

Group 6
**Job Vacancy Management & Job
Search System**

Full Design & Implementation Report

Frontend & Report (Intro, Design):	Tran Duy Tuan ((ID: 2353278))
Backend & Testing (Admin/Employer Side):	RISPAL Noa (ID: 2560118)
Backend & Testing (Job Seeker Side):	Le Huu Trieu (ID: 2353219)
Database Design & Administration:	Le Minh Anh Tuan (ID: 2453351)

Repository Link:
<https://github.com/NoaRispal/252-WebDesign-jobSearchSystem>

Contents

1	Introduction	3
2	GitHub Repository	3
2.1	Project Overview	3
2.2	Problem Statement	3
2.3	Core Features by User Role	4
2.3.1	Employer Features	4
2.3.2	Job Seeker Features	4
2.3.3	Administrator Features	4
2.4	Methodology & Engineering Approach	4
3	Database Design and Architecture	6
3.1	Project Database Overview	6
3.2	Architectural Strategy	6
3.3	Entity-Relationship Model	6
3.3.1	Core Access and Identity Tables	6
3.3.2	Reference Tables	6
3.4	Core Implementation	7
3.5	Database ERD Logic	7
4	Frontend Implementation	8
4.1	Implementation Plan Overview	8
4.1.1	Page Inventory Breakdown	8
4.2	Design System & Global Architecture	9
4.2.1	Global Shared Files	9
4.3	Phase 1: MCP & Figma Ingestion	9
4.4	Phase 2 + 3: Custom UI Augmentation	10
4.4.1	Login-Register Form (No Password Reset)	11
4.4.2	Tab-Based Admin Dashboard	14
4.4.3	Employer Job Creation Form	17
4.5	Phase 4: Dynamic Logic & DOM Manipulation	19
4.6	Phase 5: Validation & Routing	20
5	Frontend Design & Implementation Details	21
5.1	Core Page Implementations	21
5.2	UX & Navigation Logic	21
5.3	Form Structure & UI Components	21
5.4	Responsive Design Debugging Strategy	22
5.5	Client-Side Validation & Security	23

6	UML Diagrams & Architectural Rationale	24
6.1	System Use Case Diagram	24
6.1.1	Architectural Explanation	24
6.2	Activity Diagram — Job Creation Flow	25
6.2.1	Rationale: Progressive UI Chunking	26
6.3	Sequence Diagram — MVC Data Flow	26
6.3.1	MVC Separation of Concerns	27
7	Backend Implementation	28
7.1	Architecture: MVC Model	28
7.2	Authorization and Role-Based Access Control (RBAC)	28
7.3	Job Vacancy Validation & Management (Create/Delete)	29
7.4	Administrators management over references tables	31
7.5	Multi-criteria Vacancy Searching	33
7.5.1	Dynamic Filter Population	33
7.5.2	Search and Query Execution	34
7.5.3	Controller Data Validation and Fallback Logic	35
7.6	Security Measures: SQL Injection Prevention	35
8	Challenges Encountered	36
9	Conclusion	37
9.1	Summary	37
9.2	Future Works	38

1 Introduction

2 GitHub Repository

In accordance with the course requirements for version control and academic integrity, the complete source code, database schemas, and commit history for this project are maintained in a GitHub repository.

The repository includes:

- The complete MVC directory structure.
- SQL export files for the normalized database schema.
- Documentation and assets (Figma designs and reports).

Repository Link: <https://github.com/NoaRispal/252-WebDesign-jobSearchSystem>

2.1 Project Overview

The modern employment landscape is characterized by an increasing reliance on digital platforms to connect employers with prospective candidates. Despite the proliferation of commercial job portals, the underlying engineering challenges—multi-role access control, scalable data management across normalized relational schemas, and responsive user interface design—remain instructive case studies in full-stack web development.

This report documents the design, implementation, and evaluation of a **Job Vacancy Management & Job Search System**: a web-based platform that enables employers to create and manage job postings, job seekers to search and filter opportunities, and administrators to maintain the system’s reference data integrity. The application is built upon a **Model-View-Controller (MVC) architecture** using PHP, MySQL, and a custom-designed frontend.

2.2 Problem Statement

Traditional job search workflows suffer from several inefficiencies:

1. **Information asymmetry:** Job seekers must navigate fragmented listings across multiple platforms, often encountering outdated or poorly categorized postings.
2. **Limited filtering granularity:** Many platforms offer only basic keyword search, without supporting multi-dimensional filtering (by skill requirements, employment type, experience level, work arrangement, and salary range simultaneously).
3. **Administrative overhead:** Employers lack centralized dashboards for managing posting lifecycles, while system administrators have no unified interface for maintaining data consistency across reference tables.

This project addresses these challenges by implementing a single, cohesive platform with role-based access control and a comprehensive feature set tailored to three distinct user personas.

2.3 Core Features by User Role

2.3.1 Employer Features

- **Job Creation:** A structured multi-section form capturing job title, category, employment type, industry, job level, location, salary range, work arrangement, job description, and up to five required skills with individual proficiency levels.
- **Job Management Dashboard:** A centralized view displaying all postings with real-time status indicators, summary statistics, and inline action controls for editing, toggling activation status, and deleting postings.
- **Edit & Update:** The same form used for creation is repurposed for editing via a hidden `job_id` field, pre-populated with existing data by the backend controller.

2.3.2 Job Seeker Features

- **Multi-Filter Sidebar:** Nine independent filter dimensions operating with AND logic (Category, Location, Employment Type, Job Level, Work Arrangement, Date Posted, Salary Range, and Required Skills).
- **Job Detail View:** Comprehensive single-job page displaying full description, requirements, company information, and an application contact form.

2.3.3 Administrator Features

- **Content Moderation:** View all postings across all employers with the ability to remove inappropriate or policy-violating listings.
- **Reference Table CRUD:** A unified tab-based interface for managing seven lookup tables: Job Categories, Job Titles, Skills, Industries, Employment Types, Job Levels, and Salary Ranges.

2.4 Methodology & Engineering Approach

1. **Design Foundation:** A community Figma template was selected and customized to establish the system specifications. Choosing a pre-existing design foundation allowed the team to focus on technical implementation and backend architecture rather than UI creation from scratch, ensuring a professional aesthetic despite our primary expertise being in design.

2. **MVC Architecture:** The project follows a Model/View/Controller architecture allowing for a strict separation of concerns between the user interface, the business logic, and the data access layers. This modularity ensures that the **Views** are responsible entirely for the presentation layer using data provided by the controllers, while the **Controllers** orchestrate the application logic, process incoming GET/POST requests, and delegate data persistence to the models. Furthermore, the **Models** encapsulate the data access logic, securely interacting with the MySQL database through PDO prepared statements to maintain system-wide data integrity and security. This architectural choice facilitates high scalability and easier maintenance, as front-end templates can be updated independently of the underlying backend query builders and relational associations.

3 Database Design and Architecture

3.1 Project Database Overview

The database for the Job Vacancy Management and Job Search System (‘job_search_db’) is engineered to ensure high data integrity and enforce the separation of concerns via full normalization. Utilizing a robust relational schema in MySQL, the architecture guarantees efficient multi-dimensional filtering, supporting the complex querying requirements of the application’s search capabilities.

3.2 Architectural Strategy

To eliminate text-based anomalies and conform to strict data-typing protocols, the system heavily relies on structured lookup tables. All selectable fields such as locations, required skills, and job categories are stored as integer foreign keys referencing centralized dictionaries, completely avoiding plain-text or JSON data storage for critical attributes.

3.3 Entity-Relationship Model

3.3.1 Core Access and Identity Tables

The foundation of the schema establishes identity management and access permissions:

- **Roles:** Manages the permissions structure (admin, employer, job_seeker, moderator, guest).
- **Users:** Acts as the central authentication table storing encrypted credentials and establishing role-based associations via `Role_ID`.
- **Employers:** An extension of the `Users` table holding company-specific data (name, description, website), maintaining a one-to-one relationship via a unique `User_ID` with cascading deletions.

3.3.2 Reference Tables

To support the multi-dimensional search sidebar and dynamic form dropdowns, 14 dedicated reference tables govern standardized inputs:

- **Geographical Constraints:** `Countries`, `Cities`, and `Districts` implement a hierarchical cascaded location system.
- **Categorization:** `Job_Categories`, `Job_Titles`, and `Industries` define the professional scope of the posting.
- **Terms & Compensation:** `Employment_Types`, `Job_Levels`, `Work_Arrangements`, `Salary_Types`, and `Salary_Ranges`.

- **Qualifications:** Degree_Levels, Skills, and Proficiency_Levels create a controlled vocabulary for applicant requirements.

3.4 Core Implementation

The `database.sql` is the complete SQL data definition and initialization script for the application, including the creation of all core tables, reference tables, junction tables, and the insertion of test data to verify the architecture.

3.5 Database ERD Logic

The Entity-Relationship Diagram below illustrates the core relational architecture of the system. It demonstrates how users, employers, job vacancies, and required skills interact through normalized tables, utilizing primary and foreign key constraints to maintain data integrity.

Figure 1: Entity-Relationship Diagram for Job Search System

4 Frontend Implementation

4.1 Implementation Plan Overview

This implementation plan serves as the definitive engineering roadmap documenting the end-to-end process used to translate a community Figma template into a production-grade frontend. The primary goal of the plan is to build the **view layer** for the Job Vacancy Management & Job Search System, ensuring it is structurally prepared for integration into a PHP MVC architecture. The execution of this plan is strictly organized into five sequential phases, establishing a clear pipeline from static design extraction to dynamic DOM manipulation.

The Frontend development is further structured into three distinct execution phases due to the discrepancy between the provided Figma template and the strict assignment requirements:

- **Phase 1:** Build the 5 core Figma pages pixel-perfect with the help of the Figma MCP server and AI-Assisted Design-to-Code.
- **Phase 2:** Develop customized Login and Register pages, an Employer dashboard, job creation forms, and an Admin panel that initially apply only bootstrap for layout design and route planning. These views require finalized backend data structure decisions before UI implementation.
- **Phase 3:** Implement AI-Assisted flow to transfer bootstrap design into custom css to build and match the required UI/UX template's theme.

4.1.1 Page Inventory Breakdown

Page (From Figma)	File	Target Role
Homepage	index.html	Public
Jobs Listing	jobs.html	Job Seeker
Job Details	job-details.html	Job Seeker
About Us	about.html	Public
Contact Us	contact.html	Public

Table 1: Phase 1: Core pages extracted from the Figma design.

Page (Assignment Requirement)	File	Target Role
Login	login.html	All
Register	register.html	All
Employer Dashboard	employer-dashboard.html	Employer
Create/Edit Job	employer-job-form.html	Employer
Admin Dashboard	admin-dashboard.html	Administrator
Admin Reference Tables	admin-references.html	Administrator

Table 2: Phase 2 + 3: Pages required by assignment specifications.

4.2 Design System & Global Architecture

The following visual tokens were extracted directly from the Figma design source to ensure complete visual fidelity:

Token	Value	Token	Value
Primary Green	#309689	Text Gray	#616161
Primary Green Hover	#277d72	Text Light Gray	#A0A0A0
Dark BG	#18191C	Border	#E0E0E0
White	FFFFFF	Star Rating	#FFB836
Light Gray BG	#F5F7FA	Error Red	#E74C3C
Text Dark	#18191C	Fonts	Inter (400, 700)
Desktop Max Width	1200px	Mobile Breakpoint	768px

Table 3: CSS Design System Tokens.

4.2.1 Global Shared Files

To maintain a DRY (Don't Repeat Yourself) architecture, the frontend utilizes three global assets:

- **style.css:** Contains the complete CSS design system, including custom properties, CSS resets, base typography (Inter from Google Fonts), reusable components (navbars, cards, badges), responsive grid configurations, and transition animations.
- **script.js:** Manages all client-side interactivity, including the mobile hamburger menu, FAQ accordion, salary range slider, filter toggles, testimonial carousel, and client-side form validation helpers.
- **assets/:** A centralized directory for generated raster images (hero, blog, team photos) and inline SVGs utilized directly in the HTML for icons.

4.3 Phase 1: MCP & Figma Ingestion

Objective: Use the Model Context Protocol (MCP) server to programmatically read the Figma template and generate foundational HTML/CSS.

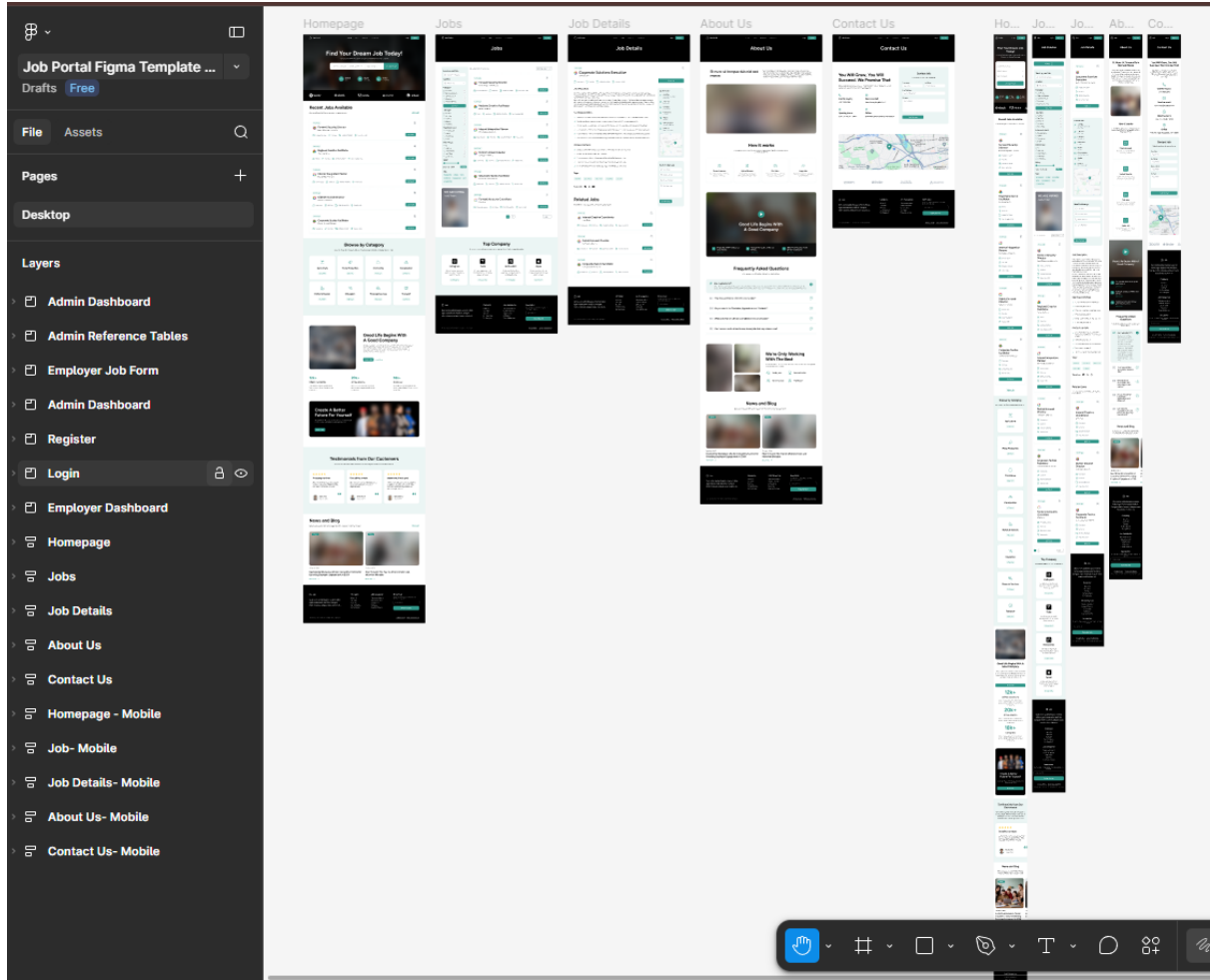


Figure 2: Figma template Design-to-code conversion

- **Environment Setup:** Install and build the Figma MCP Plugin and start the MCP Bridge Server. Connect the AI coding agent to the local port via Streaming HTTP transport.
- **Figma Layer Extraction:** Use MCP read commands to traverse the Figma layer tree and extract typography tokens, color palettes, spacing systems, and the component inventory.
- **Foundational Code Generation:** Generate a design-token CSS file (`style.css`) with CSS custom properties and semantic HTML for each page based on the Figma component hierarchy.

4.4 Phase 2 + 3: Custom UI Augmentation

Objective

Implement interactive features and structural elements that a static Figma template cannot express, using a 3-step workflow.

1. Custom built Bootstrap baseline
2. AI-assisted flow to convert into custom CSS
3. Custom CSS adjustment and refinement

4.4.1 Login-Register Form (No Password Reset)

- **Step 1 – Bootstrap Baseline:** Built responsive login and register screens with Bootstrap form controls, grid, cards, and mobile-first spacing.

The image displays two mobile application screens for a 'Job Portal'. Screen (a) is the 'Mobile Login Screen', featuring a header with the app name and a menu icon, a list of navigation links (Login, Register, Admin Demo, Employer Demo), and a central 'Welcome Back' login form. The form includes fields for email and password, a 'Remember me' checkbox, a 'Forgot password?' link, a 'Login' button, and a 'Register' link for new users. A 'Demo Accounts' section at the bottom provides test credentials. Screen (b) is the 'Mobile Register Screen', also with a header and menu icon, but with a 'Create Account' title and a sub-header 'Join and start posting or searching jobs'. It features a 'Register As' toggle for 'Job Seeker' or 'Employer', followed by fields for first name, last name, email, password, and confirm password. It includes an 'I agree to the Terms and Privacy Policy' checkbox, a 'Create Account' button, and a 'Login' link for existing users.

(a) Mobile Login Screen

(b) Mobile Register Screen

Figure 3: Mobile Login and Register form - bootstrap

Job Portal
Login
Register
Admin Demo
Employer Demo

Welcome Back

Login to continue

Email Address

Password

☐ Remember me [Forgot password?](#)

Login

No account yet? [Register](#)

Demo Accounts
Admin: admin@jobportal.com / admin123
Employer: employer@company.com / employer123

Figure 4: Login form - bootstrap

Job Portal
Login
Register
Admin Demo
Employer Demo

Create Account

Join and start posting or searching jobs

Register As

Job Seeker
Employer

First Name

Last Name

Email Address

Password

Confirm Password

☐ I agree to the Terms and Privacy Policy.

Create Account

Already have an account? [Login](#)

Figure 5: Register form - bootstrap

- **Step 2 – AI-Assisted Flow to Convert into Custom CSS:** Used AI-assisted UI refinement prompts to map Bootstrap structure into branded auth styling and role-based UX behavior (job seeker/employer/admin paths).
- **Step 3 – Custom CSS Adjustment:** Fine-tuned spacing, typography hierarchy, input focus states, button contrast, and error visibility; connected the forgot-password entry point and prepared password-reset form styling consistency.

The image shows a web application interface for a 'Job Portal'. At the top, there is a navigation bar with the 'Job Portal' logo on the left, and links for 'Home', 'Jobs', 'About Us', and 'Contact Us' in the center. On the right side of the navigation bar are 'Login' and 'Register' buttons. The main content area has a light blue background. In the center, there is a white login card with rounded corners and a subtle shadow. The card contains the 'Job Portal' logo, the heading 'Welcome Back', and the subtext 'Login to your account to continue'. Below this, there are two input fields: 'Email Address' with the placeholder 'Enter your email' and 'Password' with the placeholder 'Enter your password'. A 'Remember me' checkbox is located below the password field, and a 'Forgot password?' link is to its right. A green 'Login' button is positioned below the input fields. At the bottom of the card, there is a link 'Don't have an account? Register'. A separate box at the bottom of the card, titled 'Demo Accounts:', lists three roles with their respective email and password: 'Admin: admin@jobportal.com / admin123', 'Employer: employer@company.com / employer123', and 'Job Seeker: seeker@email.com / seeker123'.

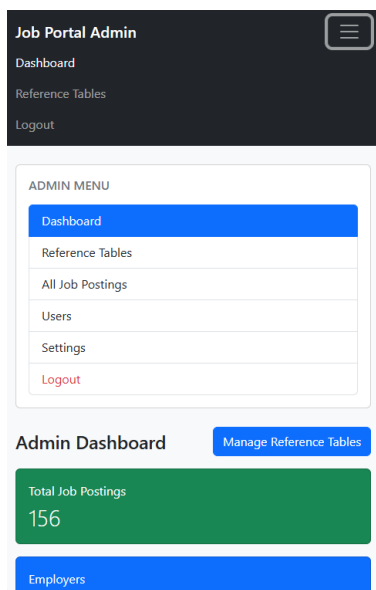
Figure 6: Login form - CSS theme conversion

The image shows a web browser displaying the 'Job Portal' registration page. The header features the 'Job Portal' logo on the left, navigation links 'Home', 'Jobs', 'About Us', and 'Contact Us' in the center, and 'Login' and 'Register' buttons on the right. The main content area is a light blue gradient. Centered on the page is a white registration form titled 'Create Account' with the subtitle 'Join Job Portal and start your journey'. The form has two tabs: 'Job Seeker' (active) and 'Employer'. Below the tabs are input fields for 'First Name', 'Last Name', 'Email Address', 'Password', and 'Confirm Password'. There is a checkbox for 'I agree to the Terms & Conditions and Privacy Policy'. A green 'Create Account' button is at the bottom of the form, with a link 'Already have an account? Login' below it.

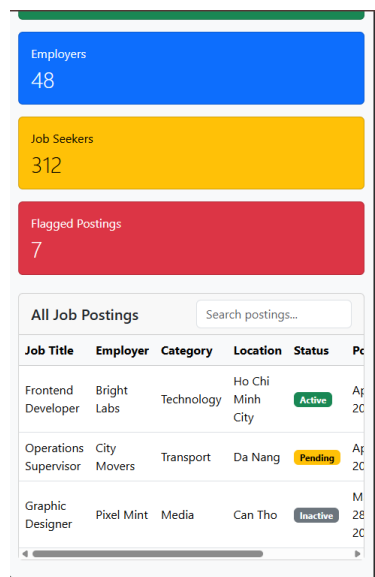
Figure 7: Register form - CSS theme conversion

4.4.2 Tab-Based Admin Dashboard

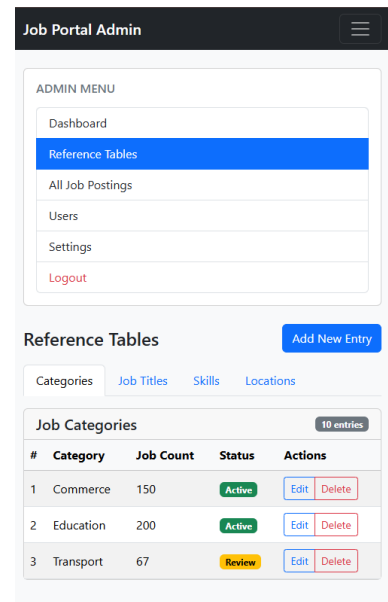
- **Step 1 – Bootstrap Baseline:** Implemented Bootstrap tab navigation and panel structure to manage reference data without full-page reload.
- **Step 2 – AI-Assisted Flow to Convert into Custom CSS:** Applied AI-guided style translation to convert default tabs, tables, and action controls into a consistent admin data-management visual language.
- **Step 3 – Custom CSS Adjustment:** Adjusted active-tab emphasis, table density, row/action alignment, modal spacing, and responsive behavior to support all 8 reference-table workflows smoothly.



(a) Admin Navigation



(b) Admin Dashboard



(c) Admin Reference Table

Figure 8: Admin Dashboard (mobile) - bootstrap

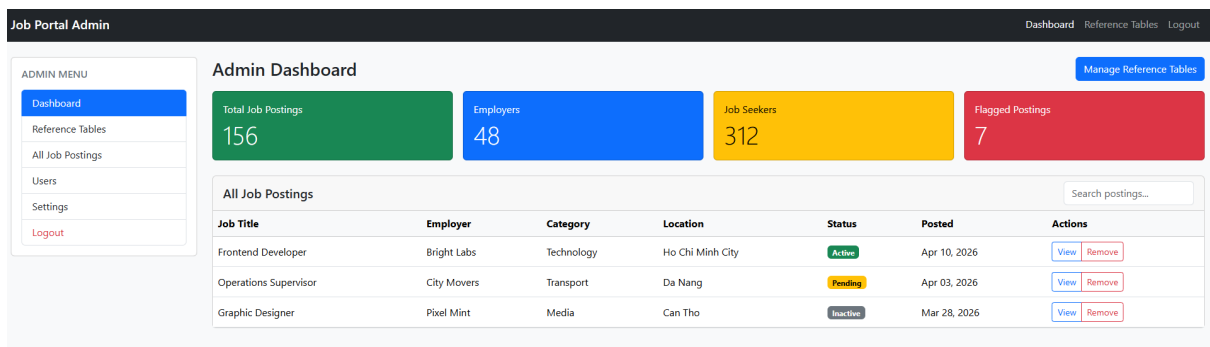


Figure 9: Admin Dashboard - bootstrap

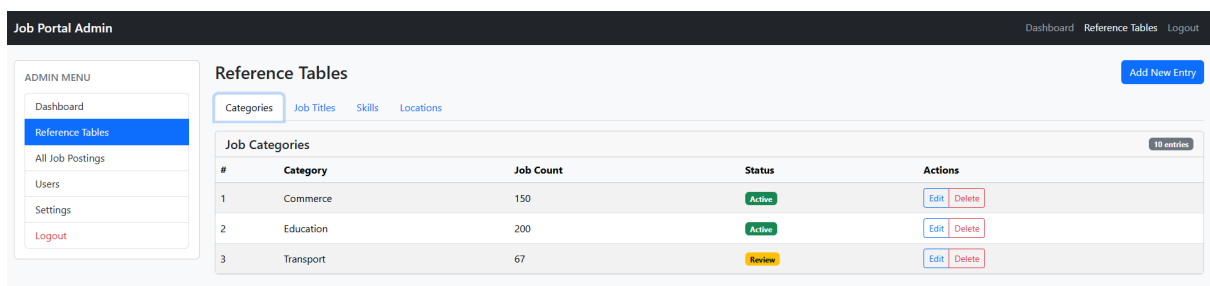


Figure 10: Admin Reference Table - bootstrap

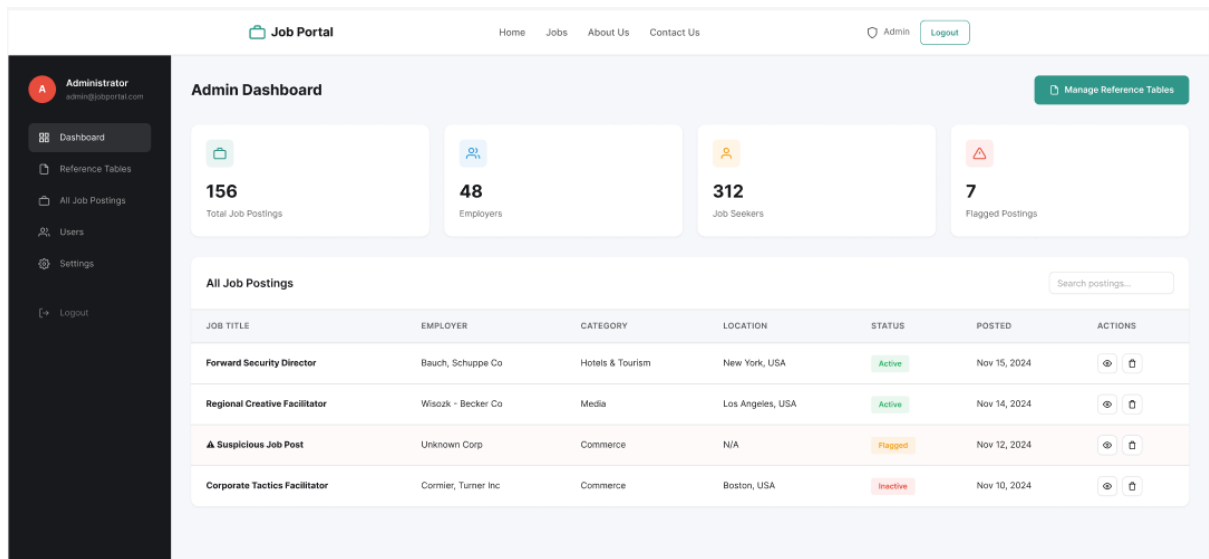


Figure 11: Admin Dashboard - CSS theme conversion

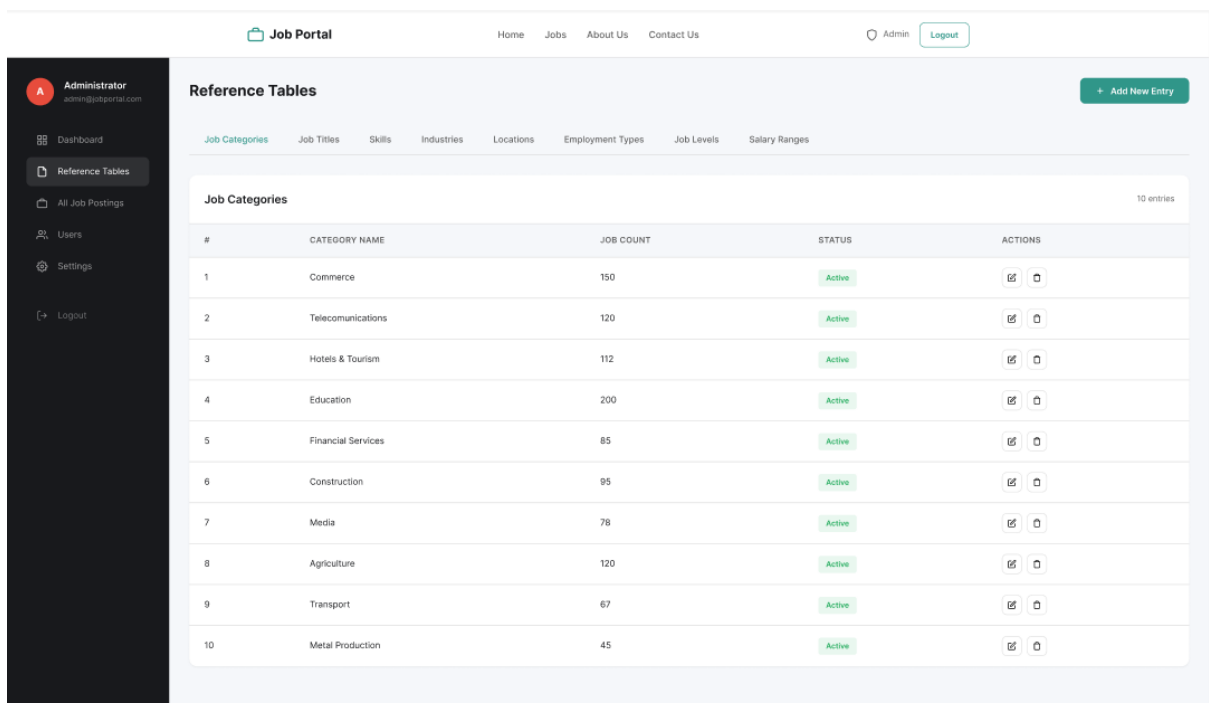


Figure 12: Admin Reference Table - CSS theme conversion

4.4.3 Employer Job Creation Form

- **Step 1 – Bootstrap Baseline:** While reusing the admin’s dashboard, tab navigation and panel structure for employer’s dashboard, created a structured employer job form with grouped sections, responsive field layout, and clear submit/cancel actions.

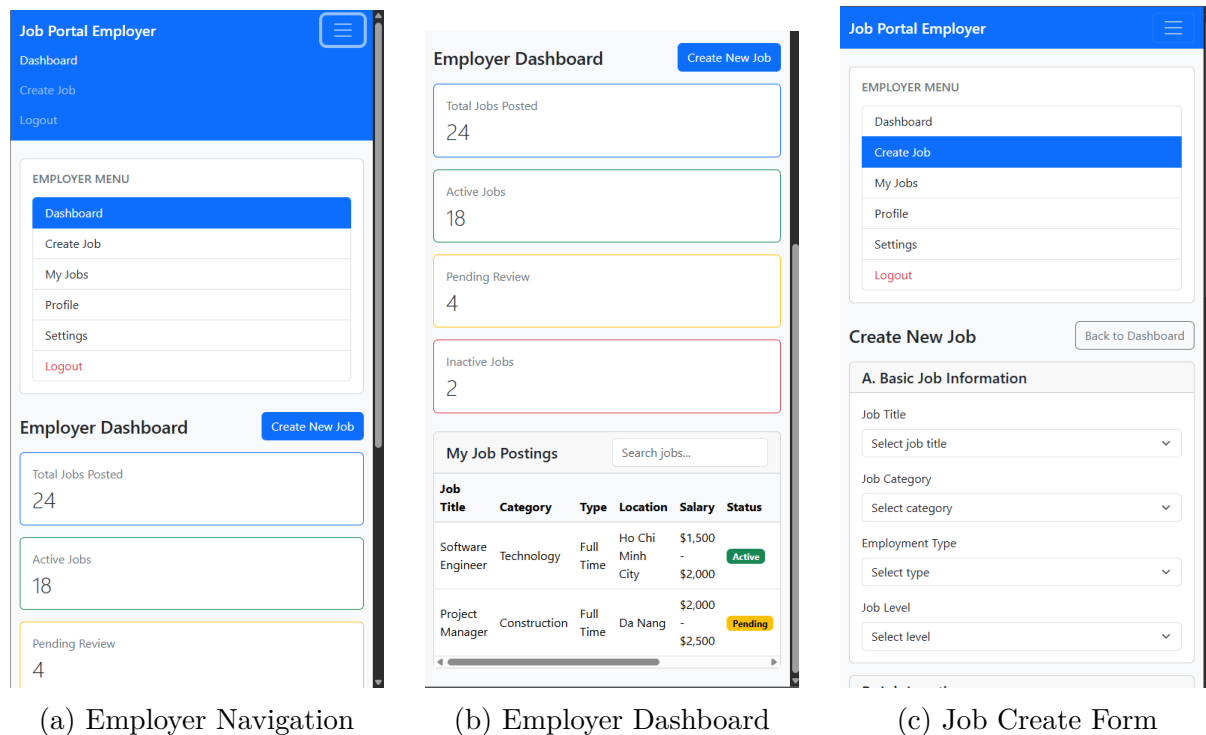


Figure 13: Employer Dashboard (mobile) - bootstrap

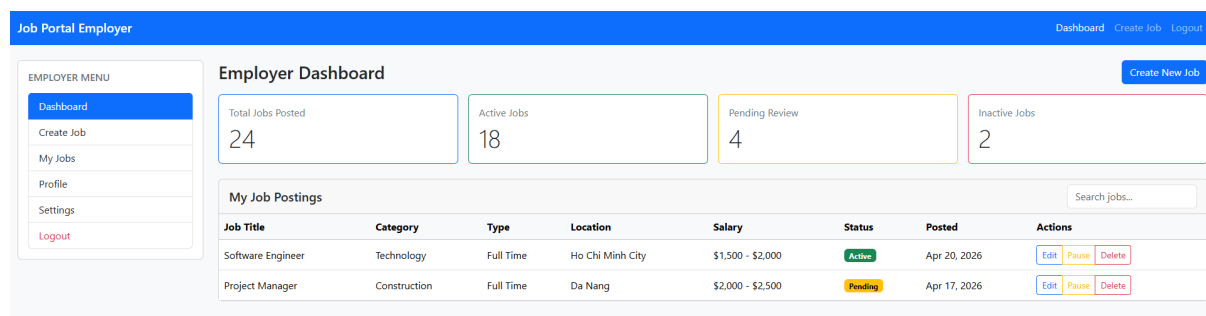


Figure 14: Employer Dashboard - bootstrap

- **Step 2 – AI-Assisted Flow to Convert into Custom CSS:** Used AI-assisted section architecture to shape the form into the required 6-part flow: Basic Info, Job Details, Location, Compensation, Description, Skills.
- **Step 3 – Custom CSS Adjustment:** Refined section headers, collapsible/visible transitions, field grouping readability, validation feedback styles, and mobile interaction spacing.

Job Portal Employer

Dashboard Create Job Logout

EMPLOYER MENU

Dashboard

Create Job

My Jobs

Profile

Settings

Logout

Create New Job

Back to Dashboard

A. Basic Job Information

Job Title

Select job title

Job Category

Select category

Employment Type

Select type

Job Level

Select level

B. Job Location

Country

Select country

City

e.g. Ho Chi Minh City

District

e.g. District 1

C. Salary and Requirements

Salary Range

Select range

Minimum Experience

Select experience

Minimum Degree

Select degree

Application Deadline

mm/dd/yyyy

D. Description

Job Description

Figure 15: Employer Job Create Form - bootstrap

Job Portal

Home Jobs About Us Contact Us

Employer Logout

Ec Employer Corp.

Dashboard

Create Job

My Jobs

Profile

Settings

Logout

Dashboard

24

Total Jobs Posted

18

Active Jobs

4

Pending Review

2

Inactive Jobs

My Job Postings

Search jobs...

Job Title	Category	Type	Location	Salary	Status	Posted	Actions
Forward Security Director	Hotels & Tourism	Full Time	New York, USA	\$40k-\$42k	Active	Nov 15, 2024	  
Regional Creative Facilitator	Media	Part Time	Los Angeles, USA	\$28k-\$32k	Active	Nov 14, 2024	  
Internal Integration Planner	Construction	Full Time	Texas, USA	\$48k-\$50k	Active	Nov 13, 2024	  
District Intranet Director	Commerce	Full Time	Florida, USA	\$42k-\$48k	Inactive	Nov 10, 2024	  
Corporate Tactics Facilitator	Commerce	Full Time	Boston, USA	\$38k-\$40k	Pending	Nov 8, 2024	  

Figure 16: Employer Dashboard - CSS theme conversion

18

Figure 17: Employer Job Creat/Edit Form - CSS theme conversion

Summary

This Phase 2 + 3 implementation follows a repeatable delivery model across all subsections: Bootstrap structure first, AI-assisted custom-CSS conversion second, and targeted custom-CSS adjustment third for usability, consistency, and backend-ready interaction patterns.

4.5 Phase 4: Dynamic Logic & DOM Manipulation

The architecture follows a **server-rendered + client-enhanced** model. PHP renders the initial HTML, and JavaScript enhances interactivity. No Single Page Application (SPA) framework was used, honoring the MVC constraints. Global interactivity is managed through `script.js`, which handles mobile hamburger toggles, FAQ accordions, salary range sliders, and client-side form validation helpers.

Pattern	Mechanism	Script Reference
Mobile navbar	<code>.active</code> class toggle	<code>initMobileNav()</code>
Admin tabs	<code>.active</code> on target panel	<code>initAdminTabs()</code>
FAQ accordion	Exclusive class toggle	<code>initFAQ()</code>
Mobile card expand	<code>.is-expanded</code> on <code><tr></code>	<code>initCardExpand()</code>

Table 4: Show/Hide JS Implementation Patterns

4.6 Phase 5: Validation & Routing

- **Client-Side Validation:** Triggered on form submit, checking [required] attributes, email regex, and password confirmation.
- **Routing Guards:** Implemented in `Public/index.php` to sanitize URL paths and enforce RBAC guards (e.g., checking `$_SESSION['role']`).
- **Delete Confirmation:** Added JS `confirm()` dialogues on all destructive actions to prevent accidental data loss.

5 Frontend Design & Implementation Details

5.1 Core Page Implementations

To comply with the assignment parameters while maintaining design fidelity, several core pages were engineered with specific architectural decisions:

- **Jobs Listing Page (`jobs.html`):** Engineered with a left-sidebar filter system featuring multi-criteria search capabilities. The filters include text input for Job Title, cascading dropdowns for Location, and checkboxes for Category, Job Type, and Experience Level. A dynamic salary range slider and date-posted radio buttons complete the sidebar. Crucially, all filters are designed to operate using AND logic, and the results can be manipulated via a sort dropdown (latest, salary, title) to fully satisfy the assignment specification.
- **Job Details Page (`job-details.html`):** Utilizes a two-column layout. The main content area displays the job description, key responsibilities, and professional skills, while the right sidebar houses an interactive "Send Us Message" form and an embedded map.
- **Interactive Forms:** All forms across the platform (including the Contact Us and Newsletter forms) are pre-configured with the appropriate `method="POST"` attributes and route to their respective PHP handler endpoints to ensure seamless backend integration.

5.2 UX & Navigation Logic

The application serves three distinct user personas, each with fundamentally different task flows. A single monolithic navigation bar would create cognitive overload. Our solution implements **context-adaptive navigation** through server-side conditional rendering.

- **Guest & Job Seeker:** Full discovery flow with login/register conversions.
- **Employer & Admin Dashboards:** Uses a **persistent sidebar** rather than a top navbar for intra-dashboard navigation. Vertical sidebars accommodate more links without horizontal overflow and create a spatial separation of concerns.

5.3 Form Structure & UI Components

All forms follow a consistent structural pattern adapted from the Figma template, providing accessibility and error-containment benefits:

```
<div class="form-group">
  <label for="field-id">Field Label</label>
  <input class="form-control" id="field-id" name="field_name" required>
```

```

    <span class="form-error">Error message text</span>
  </div>

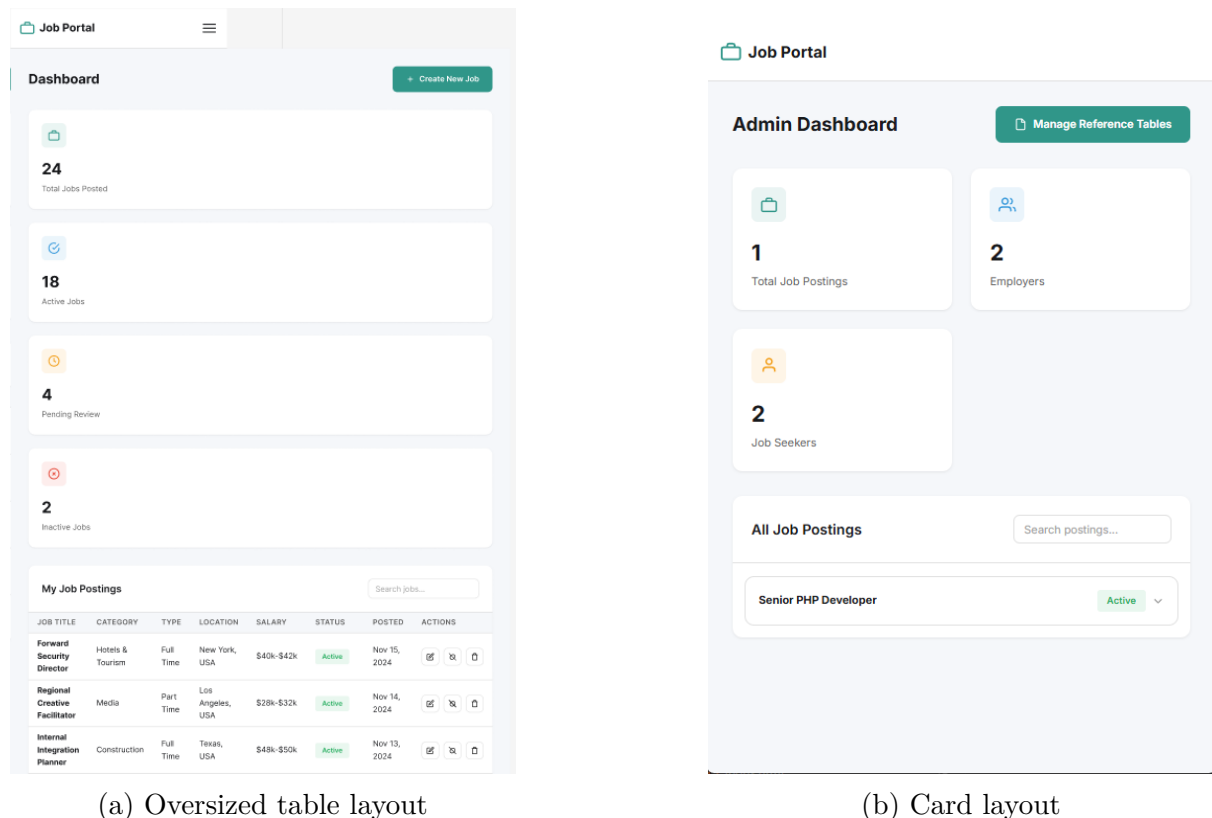
```

The **Registration Form** implements a **role-first design**: before entering personal details, the user selects their role (Job Seeker or Employer) via two large, visually distinct cards. This provides progressive disclosure and ensures a deliberate visual commitment from the user.

5.4 Responsive Design Debugging Strategy

A critical lesson learned during the AI-assisted conversion from a Bootstrap foundation to custom CSS was the limitation of table-based layouts. The generated structures proved too rigid, resulting in oversized UI elements that failed to scale and shrink responsively on smaller screens.

This specific scenario has brought up the implementation of a mobile-first enhancement approach. The most technically significant responsive pattern is the **Dashboard Table → Card Transformation**. On desktop, employer dashboards display data in a standard `<table>`. On mobile ($< 768px$), a CSS media query transforms each `<tr>` into a stacked card layout. This approach maintains a single data source—PHP loops through the data once, and CSS handles the visual presentation layer.



(a) Oversized table layout

(b) Card layout

Figure 18: Dashboard Table → Card Transformation

5.5 Client-Side Validation & Security

Client-side validation operates as a **UX enhancement layer**, not a security boundary. Its purpose is to provide immediate, inline feedback to the user before the form is submitted.

Implementation pattern:

- Empty value → triggers `.error` class on parent `.form-group`.
- Email format → evaluated via regex `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`.

Client validation is explicitly documented as insufficient; server-side validation (`filter_var`, `htmlspecialchars`) remains mandatory in the backend controllers. In order to demo the frontend role-based authentication, a `.js` has been implemented for 3 users' roles to navigate to dashboards and pages.

```
// Demo routing by email
if (email === 'admin@jobportal.com' && password === 'admin123') {
  window.location.href = 'admin-dashboard.html';
} else if (email === 'employer@company.com' && password === 'employer123') {
  window.location.href = 'employer-dashboard.html';
} else if (email === 'seeker@email.com' && password === 'seeker123') {
  window.location.href = 'index.html';
}
```


6 UML Diagrams & Architectural Rationale

6.1 System Use Case Diagram

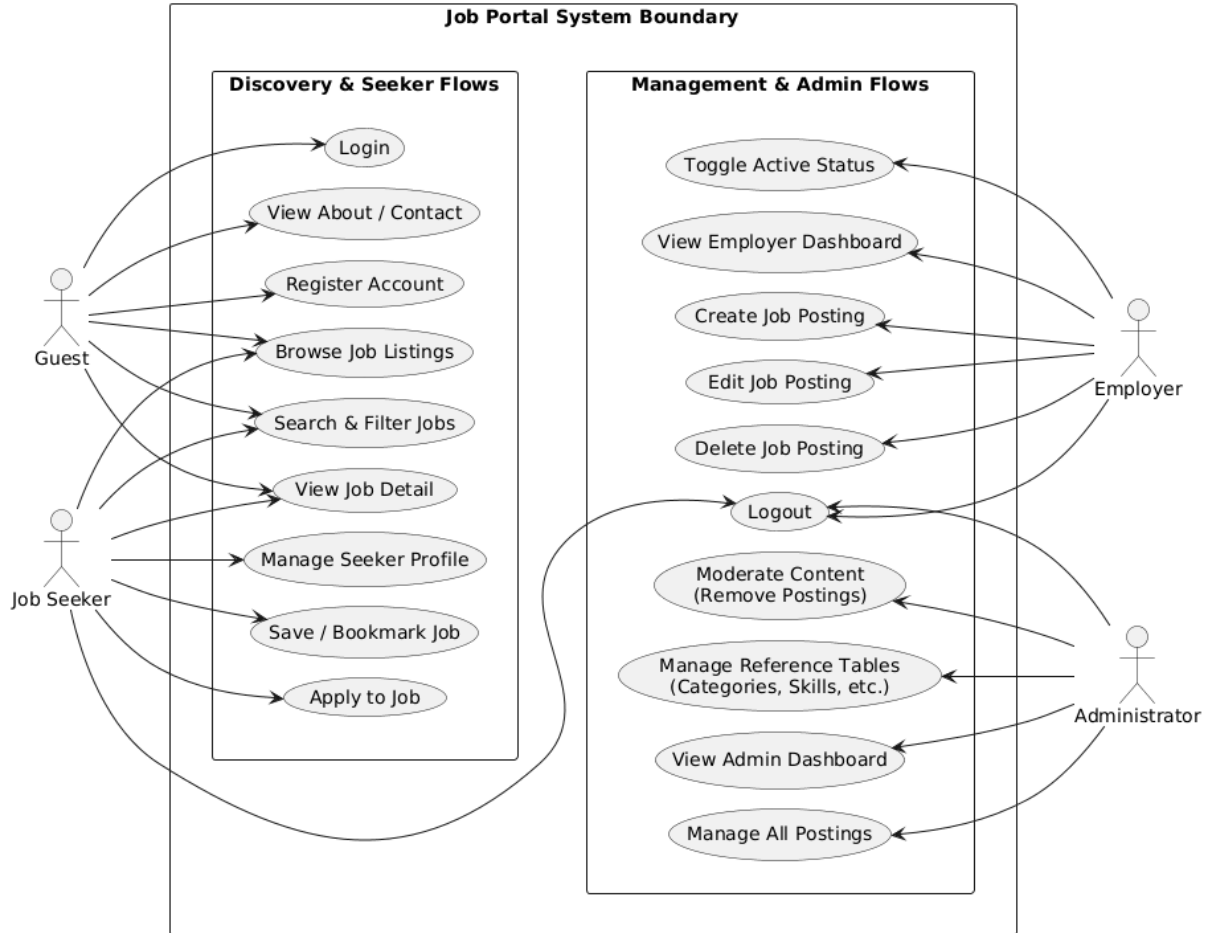


Figure 19: Overall usecase diagram

6.1.1 Architectural Explanation

The system recognizes four discrete actor archetypes. Guests and Job Seekers operate within the public discovery flow. Employers operate entirely within the job posting lifecycle via an isolated dashboard. Administrators maintain governance over system-wide job postings and reference lookup tables. Access control is enforced via server-side RBAC guards in the front controller (`index.php`), ensuring unauthorized users are redirected before any view content is exposed.

6.2 Activity Diagram — Job Creation Flow

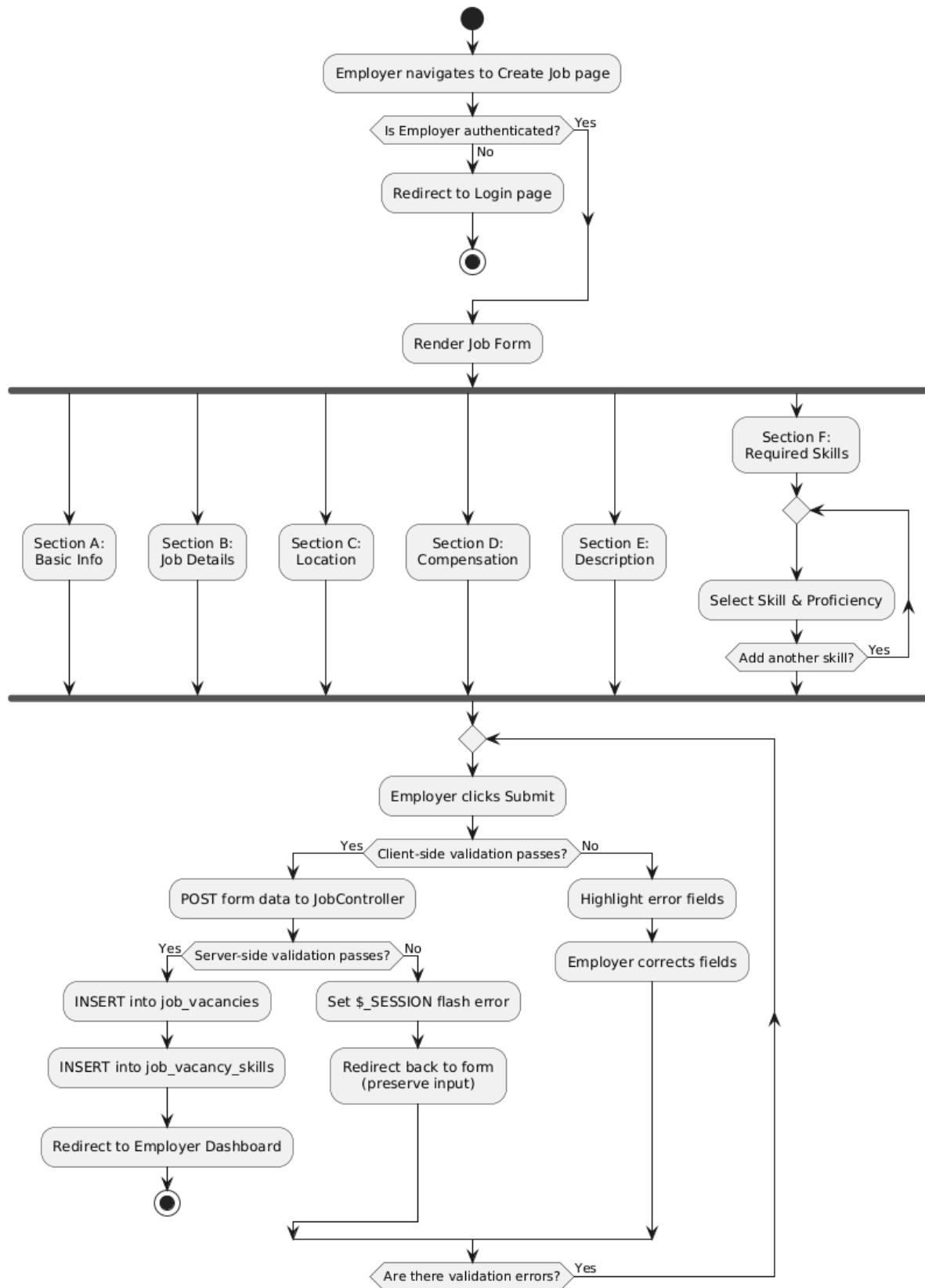


Figure 20: Job Creation Flow

6.2.1 Rationale: Progressive UI Chunking

The Job Creation form contains over 25 input fields. Presenting these fields as a single monolithic form would introduce significant cognitive overhead. The sectioning strategy applies the UX principle of progressive disclosure through chunking. Furthermore, the dual validation layers (Client-side JS + Server-side PHP) ensure that the user experience is responsive and forgiving while the data integrity guarantees remain uncompromised.

6.3 Sequence Diagram — MVC Data Flow

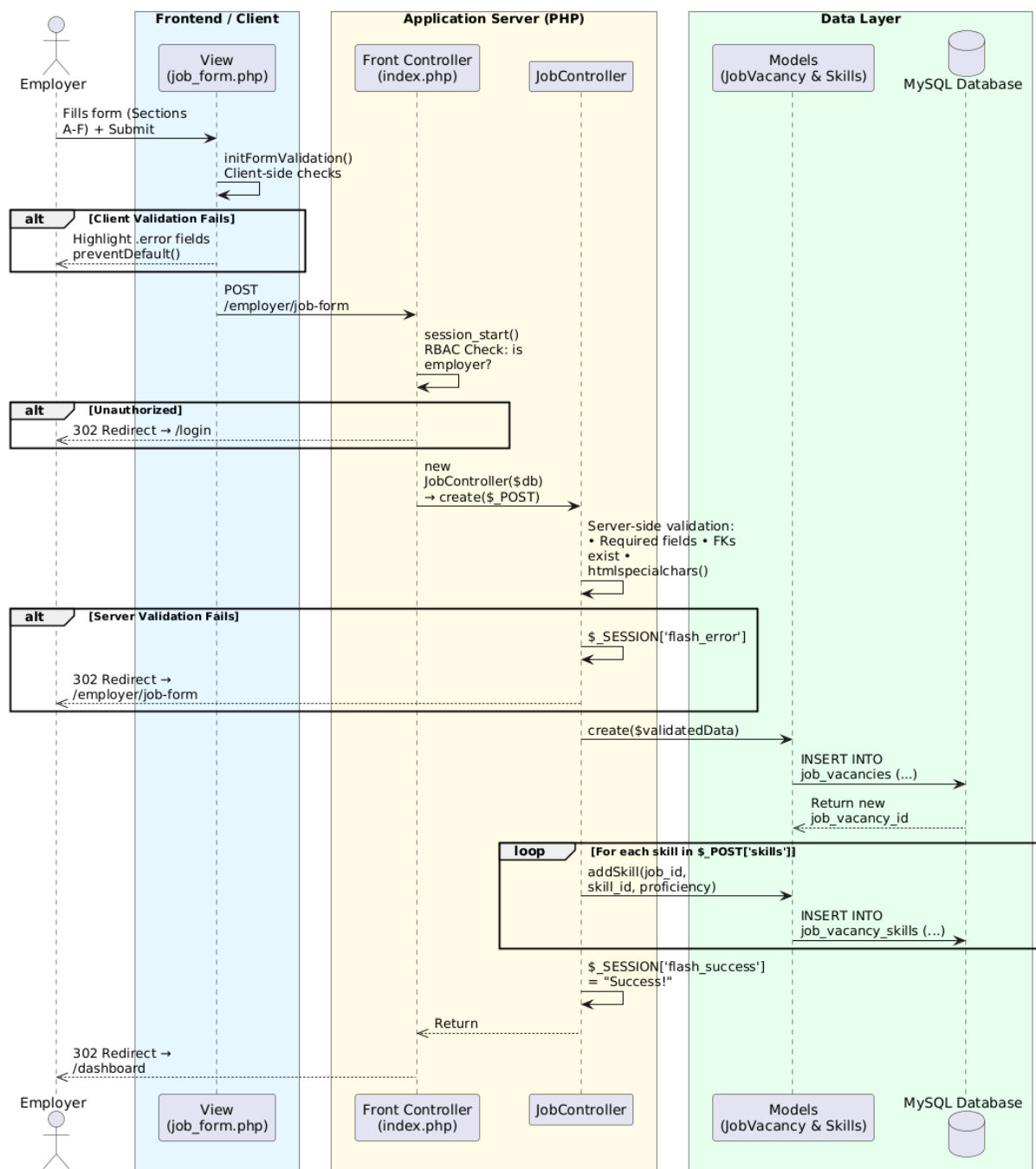


Figure 21: MVC Data Flow

6.3.1 MVC Separation of Concerns

This sequence diagram explicitly traces the lifecycle of an HTTP request through all four layers of the MVC architecture:

1. **View:** Renders HTML and executes JS validation. Has no knowledge of SQL.
2. **Router:** Handles URL parsing, session initialization, and RBAC enforcement.
3. **Controller:** Orchestrates server-side validation and business rules. Delegates persistence to the Model.
4. **Model & Database:** Encapsulates PDO prepared statements, securely inserting data into the `job_vacancies` table and returning auto-incremented IDs for junction table operations.

7 Backend Implementation

7.1 Architecture: MVC Model

The backend is structured using the Model-View-Controller (MVC) design pattern. This architectural choice improves modularity by strictly separating concerns:

- **Router:** The central entry point is `public/index.php`, which intercepts all incoming requests, sanitizes the URL parameters, and initializes the database connection. Based on the parsed URL, it delegates the request to the appropriate controller.
- **Controllers:** Handle the application logic. They process incoming GET/POST data, make calls to the models, and dictate which views are rendered.
- **Models:** Manage data access logic. The Model classes securely build query parameters and fetch data from the database.
- **Views:** Responsible entirely for the presentation layer (HTML/CSS), utilizing data provided by the controllers.

The backend implementation is divided into three primary functional areas:

1. Authorization and Role-Based Access Control
2. Job Vacancy Validation & Management (Create/Delete/Update)
3. Administrators management over jobs, users and references tables
4. Multi-criteria Vacancy Searching

7.2 Authorization and Role-Based Access Control (RBAC)

The system implements Role-Based Access Control (RBAC) to distinguish between Employers, Job Seekers, and Administrators, as outlined in the constraints.

Authentication is handled in the `AuthController::login()` function by querying the database for matching user credentials. Upon successful verification, session parameters (`$_SESSION['role']`, `$_SESSION['user_id']`) are securely assigned utilizing the live data.

RBAC enforcement occurs directly within the front controller (`public/index.php`), redirecting unauthorized users before the relevant controllers can be accessed. For instance, the **Administrator** portal strictly validates session variables:

```
1 case 'admin':  
2     // RBAC Check  
3     if (!isset($_SESSION['role']) || $_SESSION['role'] !== 'admin') {  
4         header("Location: " . $baseUrl . "/login");  
5         exit();  
6     }  
7     require_once __DIR__ . '/../controllers/adminController.php';
```

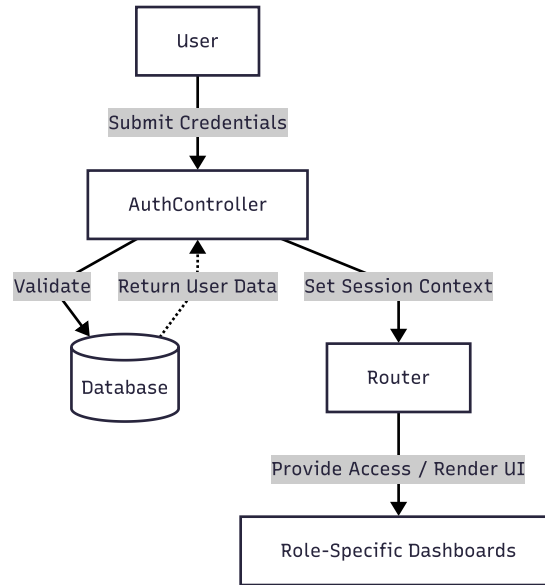


Figure 22: Authorization and Role-Based Access Flow

7.3 Job Vacancy Validation & Management (Create/Delete)

The system allows Employers to seamlessly create, edit, delete, and toggle the status of their job postings. This is managed by the back-end controller (`EmployerController`), which organizes the structured input data from dynamic multi-criteria forms before pushing it into the respective `JobVacancy` data model.

When generating a job vacancy, the controller validates the submitted `$_POST` form data and securely injects the current employer's ID. Parameter execution relies on database prepared statements in the model to prevent SQL injection. Skills are added after only and only if the job was created successfully

```

1 // controllers/employerController.php
2 public function publishJob(){
3     $employer_id =
4         $this->userModel->getEmployerDataById($_SESSION['user_id'])['Employer_ID'];
5     $jobData = $_POST
6     $jobData["emplyer_id"] = $employer_id
7     // the called method return the id from new job
8     $vacancy_id = $this->jobModel->create($jobData);
9     // Now we link skills to this job
10    if ($vacancy_id) {
11        if (!empty($_POST['skills']) && is_array($_POST['skills'])) {
12            $this->jobModel->addJobSkills($vacancy_id, $_POST['skills']);
13        }
14        $_SESSION['flash'] = "Job posted successfully!";
15        header("Location: " . $this->base_url . "/employer/dashboard");
16        exit();
17    } else {
18        $error = "Failed to create job.";
19        $_SESSION['flash'] = $error;
20        header("Location: " . $this->base_url . "/employer/dashboard");
21        exit();
22    }
23 }

```

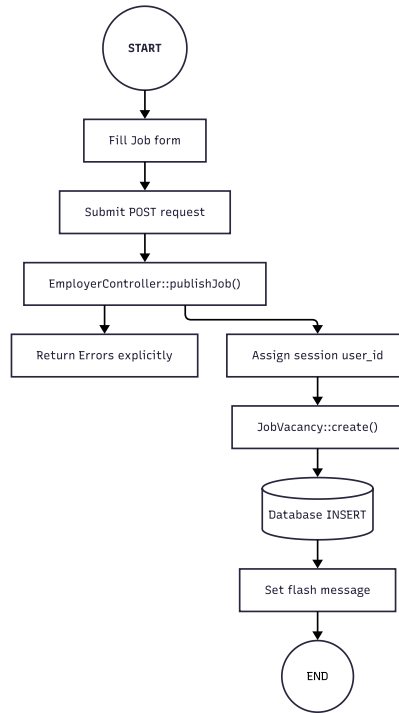


Figure 23: Job Vacancy Creation Flow

Within the JobVacancy model, creation and modification functions rely on parameterized SQL executions to interact with the normalized database safely. For example:

```

1 // models/jobVacancy.php
2 public function create($data) {
3     $query = "INSERT INTO Job_Vacancies
4               SET job_title_id=:title_id, category_id=:cat_id, location_id=:loc_id,
5                 salary_range_id=:sal_id, status='active', employer_id=:emp_id";
6     $stmt = $this->db->prepare($query);
7     $stmt->execute();
8     return $this->db->lastInsertId();
9 }
  
```

Integrity restrictions are supported at both the controller routing level and the query level. Toggling status and deletions are processed safely, ensuring that state transitions strictly affect job vacancies properly tied to verified users. Moreover, we incorporated transactions to ensure data atomicity and consistency, preventing partial updates in the event of a system failure during complex operations (like delete Job).

```

1 public function toggleStatus($job_id, $status) {
2     $new_status = 1-$status;
3     $query = "UPDATE Job_Vacancies SET Is_Active = :is_act WHERE Vacancy_ID = :id";
4     $stmt = $this->db->prepare($query);
5     return $stmt->execute(['is_act' => $new_status, 'id' => $job_id]);
6 }
7
8 public function delete($jobId, $employer_id) {
9     try {
10         // For security reasons
11         // (if there is an issue on one one the following req, we don't want the
12         // other one to succeed)
13         $this->db->beginTransaction();
  
```

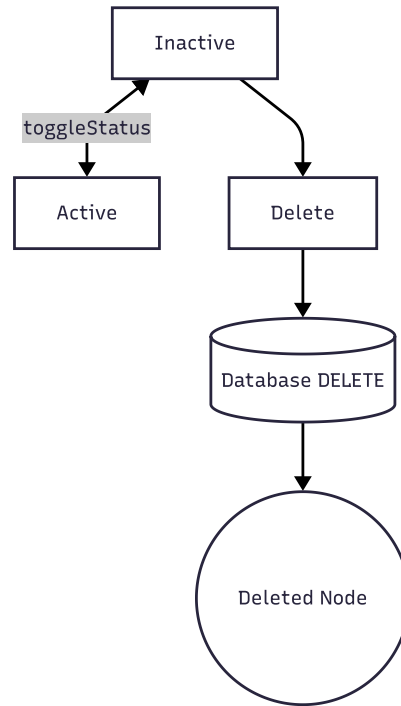


Figure 24: Job Vacancy Status & Deletion Management

```

14
15      // FK Constraints
16      $sqlSkills = "DELETE FROM Job_Vacancy_Skills WHERE Vacancy_ID = :vac_id";
17      $stmtSkills = $this->db->prepare($sqlSkills);
18      $stmtSkills->execute([':vac_id' => $jobId]);
19
20      // Delete Jobs
21      $sqlJob = "DELETE FROM Job_Vacancies WHERE Vacancy_ID = :job_id AND
Employer_ID = :emp_id";
22      $stmtJob = $this->db->prepare($sqlJob);
23      $stmtJob->execute([
24          ':job_id' => $jobId,
25          ':emp_id' => $employer_id
26      ]);
27
28      $this->db->commit();
29      return $stmtJob->rowCount() > 0; // true if deleted
30
31      } catch (PDOException $e) {
32          $this->db->rollBack();
33          die("Error deleteJob : " . $e->getMessage());
34          return false;
35      }
36  }
  
```

7.4 Administrators management over references tables

The system allows Admin to delete all job postings and/or users. The admin can also create, update or delete any value in references tables This is managed by the back-end controller (`AdminController`). Since the deletion of jobs is the same that for employers (and delete users use the same logic), in this section we will focus on managing references

tables.

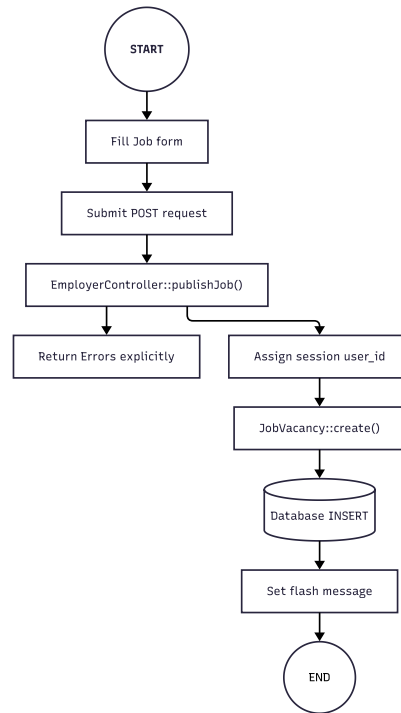


Figure 25: Job Vacancy Creation Flow

When generating a new entry in one of the references tables, the controller validates the submitted `$_POST` form data and securely injects into the database.

```

1 // controllers/employerController.php
2 public function updateLookup($action) {
3     $tableName = $_POST['table'] ?? '';
4     $value = $_POST['entry_name'] ?? '';
5     $id = $_POST['id'] ?? null;
6
7     switch ($action) {
8         case 'create':
9             $this->lookup_model->addEntry($tableName, $value);
10            $_SESSION['flash'] = "Created successfully!";
11            break;
12
13         case 'update':
14             $this->lookup_model->updateEntry($tableName, $id, $value);
15             $_SESSION['flash'] = "Updated successfully!";
16             break;
17
18         case 'delete':
19             $this->lookup_model->deleteEntry($tableName, $id);
20             $_SESSION['flash'] = "Deleted successfully!";
21             break;
22     }
23     header("Location: " . $this->base_url . "/admin/references");
24     exit();
25 }
  
```

Within the `LookupModel` model, creation and modification functions rely on parameterized SQL executions to interact with the normalized database safely. For example:

Integrity restrictions are supported at both the controller routing level and the query level. Deletions are processed safely, if some jobs are using this references, the deletions fails.

```

1 public function deleteEntry($table, $id) {
2     $table = strtolower($table);
3     if (!isset($this->columnMap[$table])) return false;
4
5     list($idCol, $nameCol) = $this->columnMap[$table];
6
7     // Check if Foreign Key is referenced in job_vacancies
8     $fkMap = [
9         'job_categories' => 'Category_ID',
10        'job_titles'      => 'Title_ID',
11        'industries'      => 'Industry_ID',
12        'employment_types' => 'Employment_Type_ID',
13        'job_levels'      => 'Level_ID',
14        'salary_ranges'   => 'Salary_Range_ID'
15    ];
16
17    if (isset($fkMap[$table])) {
18        $column = $fkMap[$table];
19        $sql = "SELECT COUNT(*) FROM Job_Vacancies WHERE $column = :id";
20        $stmt = $this->db->prepare($sql);
21        $stmt->execute(['id' => $id]);
22
23        if ($stmt->fetchColumn() > 0) {
24            return false;
25        }
26    }
27
28    if ($table === 'skills') {
29        $sql = "SELECT COUNT(*) FROM Job_Vacancy_Skills WHERE Skill_ID = :id";
30        $stmt = $this->db->prepare($sql);
31        $stmt->execute(['id' => $id]);
32        if ($stmt->fetchColumn() > 0) return false;
33    }
34
35    $query = "DELETE FROM $table WHERE $idCol = :id";
36    $stmt = $this->db->prepare($query);
37    $stmt->execute(['id' => $id]);
38    return $stmt;
39 }
40 }

```

7.5 Multi-criteria Vacancy Searching

7.5.1 Dynamic Filter Population

To adhere strictly to structured data principles, the backend dynamically populates the front-end filtering select and multi-select fields. The `JobseekerController` exposes robust methods that fetch reference table data using a generalized `JobSearchQuery` builder class. This ensures all provided options (such as Locations, Job Categories, and Required Skills) securely mirror the database schema:

```

1 class JobseekerController {
2     static public function get_categories($db) {
3         return (new JobSearchQuery($db, "Job_Categories", "Category_ID", "Category_Name")
4             )();
5     }
6 }

```

```

4 }
5
6 // Dependent dropdown filter example
7 static public function get_cities($db) {
8     if (!isset($_GET['country_id'])) return;
9     $query = new JobSearchQuery($db, "Cities", "City_ID", "City_Name");
10    $query->where_col_has_val("Country_ID", $_GET['country_id']);
11    return $query();
12 }
13 }

```

7.5.2 Search and Query Execution

The core aspect of the Job Seeker module is the ability to filter normalized job postings dynamically based on various searchable fields recursively fetched from reference tables (Job Categories, Location data, Salary, Required Skills, and Employment Types).

A dynamic query builder was designed within `models/jobSearch.php` (`JobSearchResults::handle`) to gracefully combine multiple input arrays and conditions:

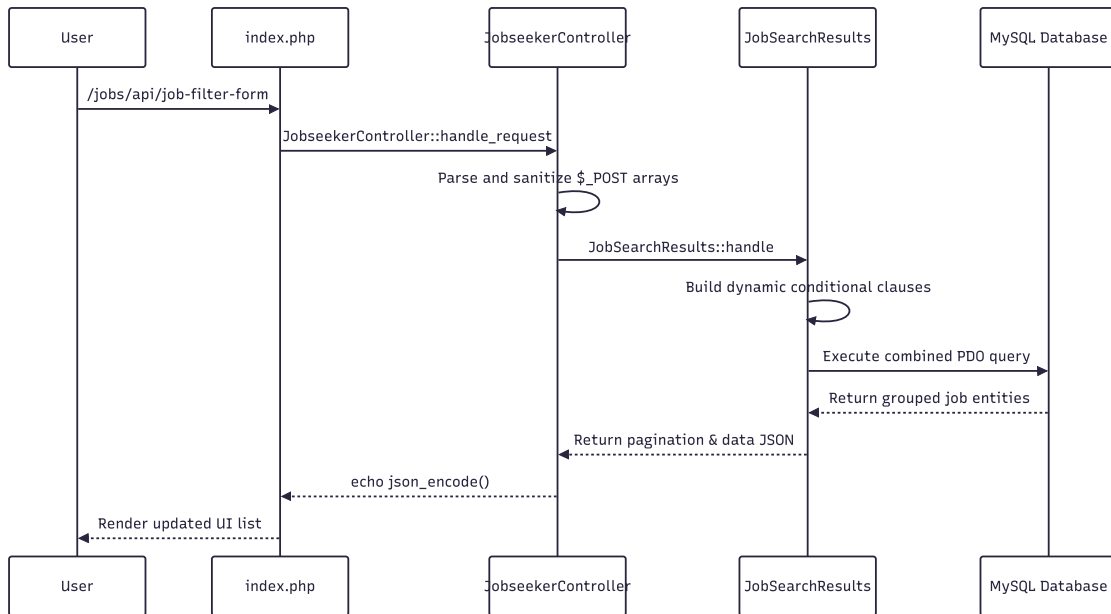


Figure 26: Job Search Sequence Diagram

In the backend `JobSearchResults::handle` model snippet below, one can observe how relational associations, ranging from basic parameter matching to handling many-to-many queries (such as Required Skills), are managed securely using PDO variables and IN clauses:

```

1 // ... (keyword and standard filter conditions omitted)
2
3 // Multi-select parameter builder
4 foreach (['category' => 'Category_ID', 'job_type' => 'Emp_Type_ID', 'job_level'
=> 'Level_ID', 'work_arrangement' => 'Arrangement_ID'] as $field => $col) {
5     if (!empty($params[$field])) {
6         $placeholders = implode(',', array_fill(0, count($params[$field]), '?'));
7
8         $cond .= " AND jv.$col IN ($placeholders)";
9     }
10 }

```

```

8         $values = array_merge($values, $params[$field]);
9     }
10 }
11
12 // Relational many-to-many skillset filter evaluation
13 if (!empty($params['skills'])) {
14     $placeholders = implode(',', array_fill(0, count($params['skills']), '?'));
15     $cond .= " AND jv.Vacancy_ID IN (
16         SELECT Vacancy_ID FROM Job_Vacancy_Skills WHERE Skill_ID IN (
17     $placeholders
18         )";
19     $values = array_merge($values, $params['skills']);
20 }

```

The system also applies pagination logic (LIMIT ? OFFSET ?), calculating offset pages efficiently based exclusively on active job records.

7.5.3 Controller Data Validation and Fallback Logic

Before the filter parameters reach the model layer, the `JobseekerController` acts as a strict gateway to process inputs and fix missing data. This ensures the model methods (like `JobSearchResults::handle`) receive consistently formatted structure without having to handle missing array keys or anomalous values:

- **Default Fallbacks:** If essential values like pagination or sorting criteria are omitted in the incoming request, the controller initializes safe default values (e.g., establishing a baseline `$params['page'] = 1` and setting the default sort to `latest`).
- **Array Deduplication & Cleanup:** Array inputs (like selected skills, categories, or keywords) are aggressively mapped over, trimmed, and deduplicated to remove empty strings and reduce redundant query conditions.
- **Null Handling:** Optional filter fields (such as District or Salary bounds) are checked. If they are missing or logically empty from `$_POST`, the controller explicitly assigns them to `null` or an empty array initialization (`[]`). This enables the underlying generic model execution to seamlessly append dynamic `WHERE` clauses only when valid parameters exist, completely avoiding undefined index errors.

7.6 Security Measures: SQL Injection Prevention

To ensure system integrity, all database interactions are heavily fortified against SQL Injection (SQLi) attacks. The application employs the following protective measures:

- **Prepared Statements:** PHP Data Objects (PDO) prepared statements are consistently utilized for all database queries. Instead of concatenating raw user inputs directly into the SQL syntax, the controller dynamically binds input arrays using parameter placeholders (?) (e.g., `$stmt = $db->prepare($sql); $stmt->execute($values);`). This ensures the database engine treats all user variables strictly as data rather than executable commands.

- **Data Sanitization and Strict Type Casting:** Incoming HTTP request variables from `$_POST` and `$_GET` are aggressively validated before any logic processing. Free-text strings are neutralized via `htmlspecialchars()`, while relational references (like category or location IDs) and numeric filters (such as salary limits) are explicitly force-cast to integers (e.g., `(int)$val`).

8 Challenges Encountered

Throughout the development of the Job Vacancy Management & Job Search System, the team faced several technical and organizational hurdles that required adaptive problem-solving and engineering trade-offs.

- **UI/UX Design for Non-Designers:** A primary challenge was delivering a professional and aesthetically pleasing interface without having professional design backgrounds. To overcome this, the team elected to customize a community Figma template. But manually translating a complex Figma design into pixel-perfect CSS is a tedious and error-prone process. By first building the core views in Bootstrap, we leveraged a familiar, robust framework to handle the layout logic and responsiveness rapidly. Using an MCP to bridge the gap allowed us to "skin" our functional Bootstrap structure with the visual identity of the Figma template. This eliminated the need for deep design expertise while ensuring the final product didn't look like a "generic" template. This approach allowed us to maintain a high standard of visual quality and focus our efforts on the complex backend implementation and relational architecture.
- **Cross-Platform Database Inconsistencies:** Collaborating in a team of four with diverse development environments (XAMPP on Windows, MAMP on macOS, and LAMPP on Linux) introduced unexpected synchronization issues. A significant hurdle was discovered regarding MySQL's handling of table names: while Linux/macOS environments are case-sensitive, XAMPP on Windows defaults to lower-case conversion for all database identifiers. This discrepancy led to "Table not found" errors that were difficult to replicate across all machines, eventually requiring the team to enforce a strict lower-case naming convention and standardized configuration files to ensure environment parity.
- **Security vs. Code Maintainability:** Implementing robust security measures, such as PDO prepared statements and strict data sanitization, was a top priority. However, we encountered a constant tension between increasing security layers and maintaining code readability. While more sophisticated security patterns could have been implemented, we found that over-engineering these measures made the codebase significantly more complex to write, review, and debug. We ultimately aimed for a balanced approach: prioritizing defense against high-risk vulnerabilities

like SQL Injection and XSS while ensuring the code remained maintainable for all team members[cite: 489, 492, 496].

- **Managing Relational Complexity:** Enforcing full normalization across 14 reference tables while managing many-to-many relationships (such as between Jobs and Skills) required meticulous query building. Ensuring that deletions and updates cascaded correctly without violating foreign key constraints—specifically in the Administrator’s management module—required extensive testing to prevent data integrity failures.

9 Conclusion

9.1 Summary

The successful implementation of the Job Vacancy Management & Job Search System fulfills all the core requirements outlined in the Assignment 2 specifications for the Web Programming course.

The technical achievements of this project are summarized as follows:

- **Advanced Database Normalization:** In strict accordance with the assignment’s constraints, the system eschews plain-text storage in favor of a fully normalized schema. By utilizing 14 reference tables and implementing a **many-to-many relationship** between job vacancies and required skills, we ensured data consistency and eliminated redundancy.
- **Architectural Integrity (MVC):** The adoption of the Model-View-Controller pattern allowed for a clean separation of concerns. This modularity was crucial for handling complex workflows, such as the dynamic job posting forms for Employers and the multi-criteria filtering sidebar for Job Seekers.
- **Secure Role-Based Access Control (RBAC):** The system successfully enforces distinct permissions for Administrators, Employers, and Job Seekers. Security is further bolstered by the systematic use of **PDO prepared statements**, neutralizing SQL injection risks and ensuring that employers can only manage their own respective postings.
- **Optimized User Experience:** Through responsive design and dynamic server-side validation, the platform provides an intuitive interface that meets the 5% usability criteria while maintaining the technical rigor required for the back-end logic.

In fine, this project serves as a practical application of structured data principles and modern web engineering. While the current scope focuses on vacancy management, the modular architecture laid down provides a solid foundation for future modules, such as an application tracking system or an automated skill-matching algorithm, should the platform be expanded beyond the current academic constraints.

9.2 Future Works

While the current system fulfills the core requirements of the Job Vacancy Management platform, several enhancements could be integrated to transition the project from a prototype to a production-ready application:

- **Real-Time Application Tracking:** Implementing a module to allow Job Seekers to upload CVs and track their application status in real-time, moving beyond the current simple contact form.
- **Automated Matching Engine:** Developing an algorithm to automatically suggest job vacancies to seekers based on the intersection of their profile skills and the required skills stored in the many-to-many junction tables.
- **Advanced Security Layers:** Integrating two-factor authentication (2FA) and more sophisticated CSRF (Cross-Site Request Forgery) protection to complement the existing SQL injection prevention measures.
- **API Extension:** Expanding the current internal JSON-encoded responses into a fully documented RESTful API, allowing for potential mobile application integration or third-party professional service connections.